

CS486/686: Introduction to Artificial Intelligence

Lecture 8c - Unsupervised Machine Learning

Jesse Hoey & Victor Zhong

School of Computer Science, University of Waterloo

March 6, 2025

Readings: Poole & Mackworth Chap. 10.3

Incomplete Data

- So far:
 - values of all attributes are known
 - learning is easy
- But many real-world problems have **hidden** variables (aka **latent** variables)
 - **Incomplete data**
 - Values of some attributes missing
- Incomplete data → **unsupervised learning**

Maximum Likelihood Learning

Recall: ML learning of Bayes net parameters for each variable V with parents $parents(V)$, and each value those parents can take on $parents(V) = \mathbf{v}$:

$$\theta_{V=true, parents(V)=\mathbf{v}} = P(V = true \mid parents(V) = \mathbf{v})$$

so that the ML learning of θ is:

$$\theta_{V=true, parents(V)=\mathbf{v}} = \frac{\text{number with } (V = true \wedge parents(V) = \mathbf{v})}{\text{number with } parents(V) = \mathbf{v}}$$

Can add pseudocounts as priors

But **what if some variable values are missing?**

Complete vs. Missing Data

For Covid diagnosis example:

- Complete data (what we used to learn from in lecture 7a)

id	Malfnction	Covid	TestB	TestA	Report	Database
1	false	false	true	true	false	false
2	false	true	true	true	true	true
3	false	true	true	true	true	true
4	false	false	false	true	false	false

...

- Incomplete (missing) data (more realistic)

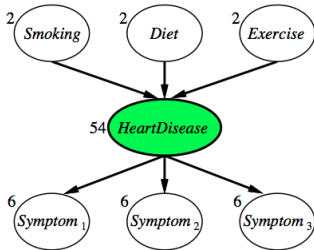
id	Malfnction	Covid	TestB	TestA	Report	Database
1	?	?	?	true	?	false
2	?	?	?	true	?	true
3	false	?	?	true	?	true
4	?	false	?	true	?	false

...

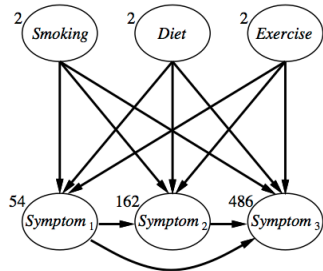
How to deal with missing data?

1. Ignore hidden variables

number of parameters shown (variables have 3 values):



(a)



(b)

2. Ignore records with missing values

- does not work with true **latent** variables (e.g. always missing)

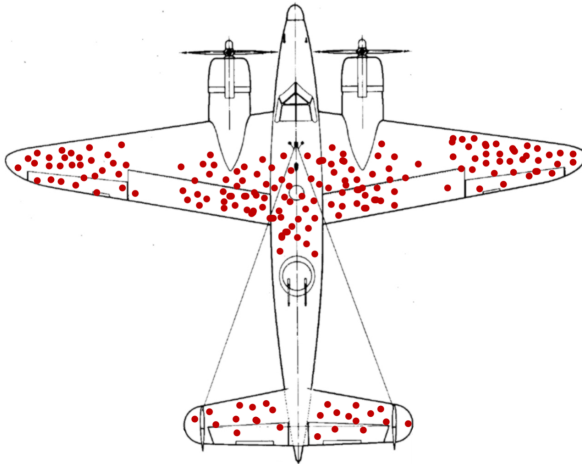
Missing Data

- You **cannot ignore** missing data unless you know it is **missing at random**.
- Often data is missing because of something **correlated with a variable of interest**.
- For example: data in a clinical trial to test a drug may be missing because:
 - the patient dies,
 - the patient dropped out because of severe side effects,
 - they dropped out because they were better, or
 - the patient had to visit a sick relative.

— ignoring some of these may make the drug look better or worse than it is.
- In general you need to **model why data is missing**.

Survivorship Bias

Bullet holes on planes returning from battle:
Where should the extra armour be installed?



Abraham Wald (WWII)

“Direct” Maximum Likelihood

3. maximize likelihood directly

Suppose $\mathbf{Z}$ is hidden and $\mathbf{E}$ is observable, with values $\mathbf{e}$

$$\begin{aligned} h_{ML} &= \arg \max_h P(\mathbf{e} \mid h) \\ &= \arg \max_h \left[\sum_{\mathbf{Z}} P(\mathbf{e}, \mathbf{Z} \mid h) \right] \\ &= \arg \max_h \left[\sum_{\mathbf{Z}} \prod_{i=1}^n P(X_i \mid \text{parents}(X_i), h)_{\mathbf{E}=\mathbf{e}} \right] \\ &= \arg \max_h \left[\log \sum_{\mathbf{Z}} \prod_{i=1}^n P(X_i \mid \text{parents}(X_i), h)_{\mathbf{E}=\mathbf{e}} \right] \end{aligned}$$

Problem: can't push log inside the sum to linearize!

Expectation-Maximization Algorithm

4. If we knew the missing values, computing h_{ML} would be easy again!

Expectation-Maximization (EM):

A). **Guess** h_{ML}

B). iterate:

- **expectation**: based on h_{ML} , compute expectation of missing values $P(\mathbf{Z} \mid h_{ML}, \mathbf{e})$
 - **maximization**: based on expected missing values, compute new estimate of h_{ML}
5. Really simple version (e.g. K-means algorithm):
- **expectation**: based on h_{ML} , compute **most likely** missing values $\arg \max_{\mathbf{Z}} P(\mathbf{Z} \mid h_{ML}, \mathbf{e})$
 - **maximization**: based on those missing values, you now have complete data, so compute new estimate of h_{ML} using ML learning as before

k -means algorithm

k -means algorithm can be used for **clustering**:

dataset of observables with input features X generated by one of a set of **classes**, C (e.g. Naïve Bayes, $C \rightarrow X$)

Inputs:

- training examples
- the number of classes, k

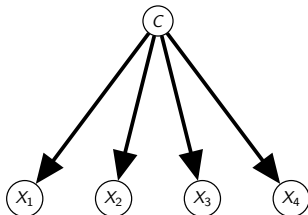
Outputs:

- a **representative value** for each input feature for each class
- an **assignment** of examples to classes

Algorithm:

1. pick k means in X , one per class, C
2. iterate until means stop changing:
 - 2.1 assign examples to k classes (e.g. as closest to current means)
 - 2.2 re-estimate k means based on assignment

Naive Bayes with 4 input features (EM)



- Suppose $k = 3$, and $\text{dom}(C) = \{1, 2, 3\}$
- each datapoint is $\{x_1, \dots, x_4\}$ (C is hidden)
- we want to find the parameters of this model using EM
- $P(C \mid X_1, X_2, X_3, X_4) \propto P(X_1 \dots X_4 \mid C)P(C)$
- can be computed if we know $P(C)$ and $P(X_i \mid C)$
- **EM idea**: based on current $P(C)$ and $P(X_i \mid C)$, compute $P(C \mid X_1 \dots X_4) \forall C \in \{1, 2, 3\}$
- use $P(C \mid X_1 \dots X_4)$ as **partial data** in ML learning

Augmented Data Method — E step – Naive Bayes

$$P(C = 1 \mid X_1 = t, X_2 = f, X_3 = t, X_4 = t) = 0.4$$

$$P(C = 2 \mid X_1 = t, X_2 = f, X_3 = t, X_4 = t) = 0.1$$

$$P(C = 3 \mid X_1 = t, X_2 = f, X_3 = t, X_4 = t) = 0.5:$$

missing data (C) $\longrightarrow$ **filled in data**

X_1	X_2	X_3	X_4
$\vdots$	$\vdots$	$\vdots$	$\vdots$
t	f	t	t
$\vdots$	$\vdots$	$\vdots$	$\vdots$



X_1	X_2	X_3	X_4	C	val
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
t	f	t	t	1	0.4
t	f	t	t	2	0.1
t	f	t	t	3	0.5
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

call this $A[X_1, \dots, X_4, C]$

M step

Compute the statistics for each feature and class:

$$M_i[X_i, C] = \sum_{X_1, \dots, X_{i-1}, X_{i+1}, \dots, X_n} A[X_1, \dots, X_n, C]$$

$$M[C] = \sum_{X_i} M_i[X_i, C]$$

$M[C]$ is unnormalized marginal.

M step

Compute the statistics for each feature and class:

$$M_i[X_i, C] = \sum_{X_1, \dots, X_{i-1}, X_{i+1}, \dots, X_n} A[X_1, \dots, X_n, C]$$

$$M[C] = \sum_{X_i} M_i[X_i, C]$$

$M[C]$ is unnormalized marginal.

Compute probabilities by normalizing:

$$P(X_i | C) = M_i[X_i, C] / M[C]$$

$$P(C) = M[C] / M$$

Pseudo-counts can also be added. (M=number of data points)

see `naivebayesml.pdf` last page

Expectation Maximization - Bonus Math!

- For a full derivation of EM, see <https://tminka.github.io/papers/minka-em-tut.pdf>. This is a condensed overview.
- **Approximate the maximum likelihood**
- Start with a **guess** h_0
- Iteratively compute:

$$h_{i+1} = \arg \max_h \sum_{\mathbf{Z}} P(\mathbf{Z}|h_i, \mathbf{e}) \log P(\mathbf{e}, \mathbf{Z}|h)$$

- **expectation**: compute $P(\mathbf{Z}|h_i, \mathbf{e})$ ("**fills in**" **missing data**)
- **maximization**: **find new** h that maximizes $\sum_{\mathbf{Z}} P(\mathbf{Z}|h_i, \mathbf{e}) \log P(\mathbf{e}, \mathbf{Z}|h)$
- can show that $P(\mathbf{e}|h_{i+1}) \geq P(\mathbf{e}|h_i)$ when computed like this

see naivebayesml.pdf last page

General Bayes Network EM

Complete data: Bayes Net Maximum Likelihood

$$\theta_{V=true, parents(V)=\mathbf{v}} = \frac{\text{number in } \mathbf{e} \text{ with } (V = true \wedge parents(V) = \mathbf{v})}{\text{number in } \mathbf{e} \text{ with } parents(V) = \mathbf{v}}$$

$parents(V)$: parents of V

Incomplete data: Bayes Net Expectation Maximization

observed variables $\mathbf{X}$ and missing variables $\mathbf{Z}$

Start with some **guess** for θ ,

E Step: Compute weights for each data $\mathbf{x}_i$ and latent variable(s) value(s) $\mathbf{z}_j$ (using e.g. variable elimination)

$$w_{ij} = P(\mathbf{z}_j | \theta, \mathbf{x}_i)$$

M Step: Update parameters:

$$\theta_{V=j, parents(V)=\mathbf{v}} = \frac{\sum_i w_{ij} | V = j \wedge parents(V) = \mathbf{v} \text{ in } \{\mathbf{x}_i, \mathbf{z}_j\}}{\sum_{ij} w_{ij} | parents(V) = \mathbf{v} \text{ in } \{\mathbf{x}_i, \mathbf{z}_j\}}$$

Belief network structure learning (I)

$$P(model|data) = \frac{P(data|model) \times P(model)}{P(data)}.$$

- A model here is a belief network.
- A bigger network can always fit the data better.
- $P(model)$ lets us encode a **preference for smaller networks** (e.g., using the description length).
- You can **search over network structure** looking for the most likely model.

Belief network structure learning (II)

- can do **independence tests** to determine which features should be the parents
- just because features do not give information individually, does not mean they will not give information in combination
- ideal: **Search over total orderings** of variables

Autoencoders - Overview

- A representation learning algorithm
- Learn to map examples to low-dimensional representation

Autoencoders - Components

2 main components

1. *Encoder* $e(x)$: maps x to low-dimensional representation $\hat{z}$
2. *Decoder* $d(\hat{z})$: maps $\hat{z}$ to its original representation x

Autoencoder implements $\hat{x} = d(e(x))$

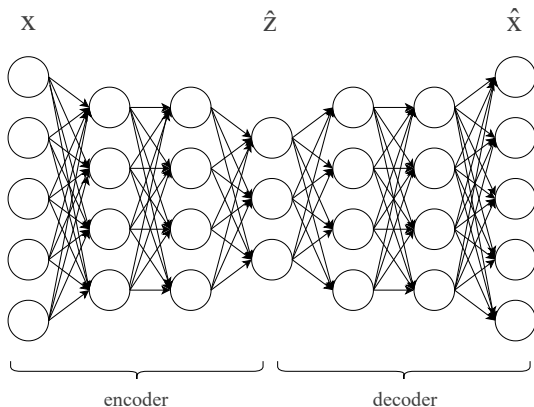
- $\hat{x}$ is the *reconstruction* of original input x
- Encoder and decoder learned such that $\hat{z}$ contains as much information about x as needed to reconstruct it

Minimize sum of squares of differences between input and prediction:

$$E = \sum_i (x_i - d(e(x_i)))^2$$

Deep Neural Network Autoencoders

- Good for complex inputs
- e and d are feedforward neural networks, joined in series
- Train with backpropagation



Generative Adversarial Networks - Overview

a.k.a. GANs

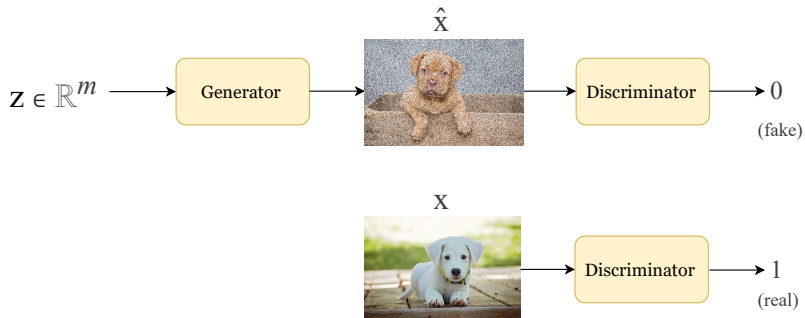
- A generative unsupervised learning algorithm
- Goal is to generate unseen examples that look like training examples

GANs - Components

GANs are actually a pair of neural networks:

- *Generator* $g(z)$: Given vector z in latent space, produces example x drawn from a distribution that approximates the true distribution of training examples
 z usually sampled from a Gaussian distribution
- *Discriminator* $d(x)$: A classifier that predicts whether x is real (from training set) or fake (made by g)

GANs - Illustrative Example



GANs - Training

GANs are trained with a minimax error:

$$E = \mathbb{E}_x[\log(d(x))] + \mathbb{E}_z[\log(1 - d(g(z)))]$$

- Discriminator tries to maximize E
 - for x from the training set, $d(x) \rightarrow 1$,
 - for x from the generator (based on z), $d(x) \rightarrow 0$
- Generator tries to minimize E - tries to fool d
 - for x from the training set, $d(x) \rightarrow 0$,
 - for x from the generator (based on z), $d(x) \rightarrow 1$

GANs - Training

GANs are trained with a minimax error:

$$E = \mathbb{E}_x[\log(d(x))] + \mathbb{E}_z[\log(1 - d(g(z)))]$$

- Discriminator tries to maximize E
 - for x from the training set, $d(x) \rightarrow 1$,
 - for x from the generator (based on z), $d(x) \rightarrow 0$
- Generator tries to minimize E - tries to fool d
 - for x from the training set, $d(x) \rightarrow 0$,
 - for x from the generator (based on z), $d(x) \rightarrow 1$

After convergence:

- g should be producing realistic images
- d should output $\frac{1}{2}$, indicating maximal uncertainty

Next

- Decision-Making (Poole & Mackworth chapter 12.1-12.3,12.5)
- Reinforcement Learning (Poole & Mackworth chapter 13.1-12.9)